

	Politechnika Bydgoska im. Jana i Jędrzeja Śniadeckich Wydział Telekomunikacji, Informatyki i Elektrotechniki Nowoczesne Technologie Przetwarzania Danych Laboratorium 11 Temat: Przetwarzanie strumieniowe danych w Apache Spark Structured Streaming	
---	---	---

Sprawozdanie, Marcin Gregrowicz, semestr VI, grupa nr 1

Cel ćwiczenia

Celem ćwiczenia jest praktyczne poznanie przetwarzania strumieniowego danych w Apache Spark Structured Streaming; utworzenie strumieniowego DataFrame; wykonanie transformacji i agregacji na danych napływających w czasie; wykorzystanie okien czasowych, watermarkingu i checkpointingu; zapis wyników strumienia do konsoli oraz plików.

Link do repozytorium Git:

Zadanie 1: Przygotowanie środowiska

Kod tego zadania znajduje się w pliku Zad01.py.

W tym laboratorium, tak jak w poprzednich, skorzystałem z wersji PySparka 3.5.4

```
Pomyślnie uruchomiono sesję Spark!  
Wersja Spark/PySpark: 3.5.4
```

Zadanie 2: Strumieniowe wczytywanie danych

Dane zostały wygenerowane z wykorzystaniem skryptu Generator.py (dodaje on automatycznie nowe pliki do folderu wejściowego, tak jak wspomniane jest w zadaniu 3).

Kod tego zadania znajduje się w pliku Zad02.py.

Po podstawowym czyszczeniu danych, wyświetliłem schemat stworzonego DataFrame:

```
Schemat przetworzonego DataFrame:
root
|-- event_time: timestamp (nullable = true)
|-- user_id: string (nullable = true)
|-- category: string (nullable = true)
|-- amount: double (nullable = true)
|-- status: string (nullable = true)
```

Zadanie 3: Transformacje i agregacje strumieniowe

Kod tego zadania znajduje się w pliku Zad03.py

Uruchomiłem ten skrypt, a następnie uruchomiłem generator danych, który tworzył nowy plik co 20 sekund.

```
Batch: 0
-----
+-----+-----+-----+-----+
| category|events_count|total_amount|total_vat|
+-----+-----+-----+-----+
| books|      19|    4389.79|   1009.68|
|electronics|      17|    4183.25|    962.15|
| sports|      15|    4617.36|   1062.0|
| clothing|      17|    3505.03|    806.17|
| home|      18|    3504.14|    805.96|
+-----+-----+-----+-----+

Batch: 1
-----
+-----+-----+-----+-----+
| category|events_count|total_amount|total_vat|
+-----+-----+-----+-----+
| books|      20|    4475.54|   1029.4|
|electronics|      19|    4459.29|   1025.64|
| sports|      15|    4617.36|   1062.0|
| clothing|      18|    3914.12|    900.26|
| home|      20|    4271.19|    982.38|
+-----+-----+-----+-----+
```

Jak widać na powyższym zrzucie ekranu, Spark zdołał przetworzyć nowo dodane dane bez resetu programu, widać różnicę pomiędzy batchem 0 i 1.

W kontekście tego laboratorium, tryb complete przy każdym nowym batchu przelicza i wyświetla całą tabelę wynikową ze wszystkimi dotychczasowymi danymi od początku działania aplikacji, co jest idealne do globalnych agregacji, podczas gdy tryb update wypisuje w konsoli wyłącznie te wiersze (okna czasowe lub kategorie), w których w danej chwili nastąpiła jakakolwiek zmiana, pozostawiając tabelę pustą w przypadku braku nowych modyfikacji. Z kolei tryb append przekazuje na wyjście tylko zupełnie nowe,

sfinalizowane rekordy i w przypadku korzystania z okien czasowych z watermarkiem zapisuje podsumowanie danego okna na dysk dopiero wtedy, gdy zostanie ono ostatecznie zamknięte i żaden opóźniony rekord już do niego nie trafi, gwarantując nienaruszalność raz zapisanych danych.

W tym zadaniu skorzystałem z complete, by wyświetlić wszystkie dane.

Zadanie 4: Okna czasowe i watermarking

W kontekście tego laboratorium, czas zdarzenia to moment, w którym transakcja faktycznie się odbyła (data wygenerowana przez skrypt i zapisana w pliku CSV). Czas przetwarzania to czas na zegarze komputera w momencie, gdy Spark przetwarza te dane. W streamingu dane grupowane są niemal zawsze po czasie zdarzenia, gdyż jest on odporny na opóźnienia sieciowe.

Aplikacja została przetestowana na danych przychodzących w poprawnej kolejności oraz opóźnionych (na poniższych zrzutach te z 05-27). Dane miały też różne kategorie i wartości.

Okno stałe:

```
Batch: 0
```

window	category	events_count
{2026-05-29 00:20:00, 2026-05-29 00:30:00}	books	16
{2026-05-29 00:20:00, 2026-05-29 00:30:00}	home	16
{2026-05-27 14:10:00, 2026-05-27 14:20:00}	electronics	5
{2026-05-27 14:10:00, 2026-05-27 14:20:00}	books	6
{2026-05-27 14:00:00, 2026-05-27 14:10:00}	books	30
{2026-05-27 14:10:00, 2026-05-27 14:20:00}	home	9
{2026-05-27 14:00:00, 2026-05-27 14:10:00}	clothing	35
{2026-05-29 00:20:00, 2026-05-29 00:30:00}	sports	19
{2026-05-27 14:00:00, 2026-05-27 14:10:00}	home	36
{2026-05-27 14:10:00, 2026-05-27 14:20:00}	sports	10
{2026-05-27 14:00:00, 2026-05-27 14:10:00}	sports	26
{2026-05-27 14:00:00, 2026-05-27 14:10:00}	electronics	49
{2026-05-29 00:20:00, 2026-05-29 00:30:00}	electronics	22
{2026-05-27 14:10:00, 2026-05-27 14:20:00}	clothing	6
{2026-05-29 00:20:00, 2026-05-29 00:30:00}	clothing	13

```
Batch: 1
```

window	category	events_count
{2026-05-29 00:50:00, 2026-05-29 01:00:00}	sports	1
{2026-05-29 00:50:00, 2026-05-29 01:00:00}	home	4
{2026-05-29 00:50:00, 2026-05-29 01:00:00}	electronics	3
{2026-05-29 00:50:00, 2026-05-29 01:00:00}	clothing	2
{2026-05-29 00:50:00, 2026-05-29 01:00:00}	books	2

Okno przesuwane:

```
Batch: 0
```

window	category	events_count
{2026-05-29 00:20:00, 2026-05-29 00:30:00}	books	16
{2026-05-29 00:20:00, 2026-05-29 00:30:00}	home	16
{2026-05-29 00:25:00, 2026-05-29 00:35:00}	clothing	6
{2026-05-29 00:25:00, 2026-05-29 00:35:00}	electronics	8
{2026-05-27 14:10:00, 2026-05-27 14:20:00}	electronics	5
{2026-05-27 14:10:00, 2026-05-27 14:20:00}	books	6
{2026-05-27 14:00:00, 2026-05-27 14:10:00}	books	30
{2026-05-29 00:15:00, 2026-05-29 00:25:00}	sports	12
{2026-05-27 14:10:00, 2026-05-27 14:20:00}	home	9
{2026-05-27 14:05:00, 2026-05-27 14:15:00}	sports	36
{2026-05-27 14:00:00, 2026-05-27 14:10:00}	clothing	35
{2026-05-29 00:25:00, 2026-05-29 00:35:00}	home	4
{2026-05-29 00:20:00, 2026-05-29 00:30:00}	sports	19
{2026-05-29 00:25:00, 2026-05-29 00:35:00}	sports	7
{2026-05-29 00:15:00, 2026-05-29 00:25:00}	electronics	14
{2026-05-29 00:25:00, 2026-05-29 00:35:00}	books	3
{2026-05-27 14:05:00, 2026-05-27 14:15:00}	electronics	54
{2026-05-27 14:00:00, 2026-05-27 14:10:00}	home	36
{2026-05-27 14:10:00, 2026-05-27 14:20:00}	sports	10
{2026-05-27 14:00:00, 2026-05-27 14:10:00}	sports	26

```
Batch: 1
```

window	category	events_count
{2026-05-29 00:55:00, 2026-05-29 01:05:00}	clothing	2
{2026-05-29 00:50:00, 2026-05-29 01:00:00}	sports	2
{2026-05-29 00:55:00, 2026-05-29 01:05:00}	electronics	4
{2026-05-29 00:50:00, 2026-05-29 01:00:00}	home	4
{2026-05-29 00:50:00, 2026-05-29 01:00:00}	electronics	4
{2026-05-29 00:50:00, 2026-05-29 01:00:00}	clothing	2
{2026-05-29 00:55:00, 2026-05-29 01:05:00}	home	4
{2026-05-29 00:55:00, 2026-05-29 01:05:00}	sports	2
{2026-05-29 00:55:00, 2026-05-29 01:05:00}	books	3
{2026-05-29 00:50:00, 2026-05-29 01:00:00}	books	3

W oknie statym dane z danej minuty trafiają tylko do jednego, sztywnego, 10-minutowego przedziału, ponieważ okna te nie nakładają się na siebie. Natomiast w oknie przesuwanym te same dane trafiają do dwóch różnych okien jednocześnie (np. kończącego się o 01:00 i zaczynającego o 00:55), ponieważ okna nakładają się na siebie o wyznaczony krok (5 minut).

Zadanie 5: Zapis wyników i checkpointing

Kod tego ćwiczenia znajduje się w plikach Zad05.py oraz Zad05_odczyt.py.

Zapisane wyniki znajdują się w folderze data/output_stream.

Po ponownym uruchomieniu aplikacji strumieniowej silnik Spark odczytał stan z katalogu checkpoints/lab11, co zasygnalizował poniższym komunikatem. Aplikacja nie przetworzyła historycznych plików wejściowych ponownie i nie zdublowała rekordów w bazie Parquet

```
26/05/29 20:09:37 WARN HDFSBackedStateStoreProvider: The state for version 4 doesn't exist in loadedMaps. Reading snapshot file and delta files if needed...Note that this is normal for the first batch of starting query.
```

Wyświetlenie zapisanych plików:

```
[SUCCES] Liczba zapisanych wierszy w plikach Parquet: 20
Aktualna zawartość zapisana na dysku przez strumień:
+-----+-----+-----+-----+
|window_start|window_end|category|events_count|
+-----+-----+-----+-----+
|2026-05-27 14:10:00|2026-05-27 14:20:00|electronics|5|
|2026-05-29 00:50:00|2026-05-29 01:00:00|electronics|12|
|2026-05-27 14:00:00|2026-05-27 14:10:00|electronics|49|
|2026-05-29 00:20:00|2026-05-29 00:30:00|electronics|22|
|2026-05-27 14:00:00|2026-05-27 14:10:00|clothing|35|
|2026-05-29 00:50:00|2026-05-29 01:00:00|clothing|7|
|2026-05-27 14:10:00|2026-05-27 14:20:00|clothing|6|
|2026-05-29 00:20:00|2026-05-29 00:30:00|clothing|13|
|2026-05-29 00:50:00|2026-05-29 01:00:00|sports|11|
|2026-05-29 00:20:00|2026-05-29 00:30:00|sports|19|
|2026-05-27 14:10:00|2026-05-27 14:20:00|sports|10|
|2026-05-27 14:00:00|2026-05-27 14:10:00|sports|26|
|2026-05-29 00:20:00|2026-05-29 00:30:00|books|16|
|2026-05-27 14:10:00|2026-05-27 14:20:00|books|6|
|2026-05-27 14:00:00|2026-05-27 14:10:00|books|30|
|2026-05-29 00:50:00|2026-05-29 01:00:00|books|7|
|2026-05-29 00:20:00|2026-05-29 00:30:00|home|16|
|2026-05-29 00:50:00|2026-05-29 01:00:00|home|9|
|2026-05-27 14:10:00|2026-05-27 14:20:00|home|9|
|2026-05-27 14:00:00|2026-05-27 14:10:00|home|36|
+-----+-----+-----+-----+
```

Przetwarzanie Batch (wsadowe) a Streaming (strumieniowe):

Przetwarzanie Batch: Polega na przetwarzaniu z góry określonych, skończonych i statycznych zbiorów danych (np. plików historycznych, baz danych z poprzedniego dnia). Cały proces ma wyraźny początek i koniec, aplikacja uruchamia się, wczytuje pełen blok danych, wykonuje obliczenia i zapisuje wynik końcowy, po czym kończy działanie. Kluczowym kryterium jest tu wysoka przepustowość, a nie czas odpowiedzi.

Przetwarzanie Streaming: Odnosi się do ciągłego, nieprzerwanego przetwarzania danych, które napływają w czasie rzeczywistym z nieskończonych źródeł. Aplikacja działa bez przerwy w pętli tzw. mikro-batchów, przetwarzając nowe porcje danych w miarę ich pojawiania się. Głównym celem jest minimalizacja opóźnień i dostarczanie aktualnych wyników na bieżąco.

Różnice pomiędzy trybami append, update i complete zostały opisane w zadaniu 3.

Checkpointing a zwykły zapis plików wynikowych:

Zwykły zapis plików wynikowych: Odpowiada wyłącznie za fizyczne zrzucenie przetworzonych w danym momencie danych z pamięci RAM na trwały nośnik. Zwykły zapis nie posiada jednak wiedzy o historii strumienia ani o tym, które pliki wejściowe zostały już przetworzone. W przypadku awarii lub restartu aplikacja nie wie, w którym miejscu przerwała pracę, co prowadzi do ponownego przetwarzania tych samych danych (duplikacji) lub ich utraty.

Checkpointing: To zaawansowany mechanizm zapewniający odporność na awarie w systemach strumieniowych. Oprócz samych danych, w niezależnym katalogu Spark zapisuje metadane transakcyjne, aktualne wskaźniki źródła danych oraz stan agregacji w oknach czasowych. Dzięki temu, po nagłym zatrzymaniu lub awarii systemu, aplikacja po restarcie odczytuje stan checkpointu i wznowia pracę dokładnie od momentu przerwania, gwarantując semantykę *exactly-once* bez ryzyka powstawania duplikatów.